

P1018R11

C++ Language Evolution status pandemic edition 2021/05

Published Proposal, 2021-06-01

This version:

<http://wg21.link/P1018r11>

Author:

[JF Bastien](#) (Woven Planet)

Audience:

WG21, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1018r11.bs

Abstract

This paper is a collection of items that the C++ Language Evolution group has worked on in the latest meeting, their status, and plans for the future.

Table of Contents

- 1 **Executive summary**
- 2 **Paper of note**
- 3 **Tentatively ready papers**
- 4 **Issue Processing**
- 5 **Poll**
 - 5.1 P0847r6 Deducing `this`
 - 5.2 P0849r7 `auto(x)`: `DECAY_COPY` in the language
 - 5.3 P2242r2 Non-literal variables (and labels and `goto`s) in `constexpr` functions
 - 5.4 CWG728 Restrictions on local classes
 - 5.5 CWG944 `reinterpret_cast` for all types with the same size and alignment
 - 5.6 CWG1451 Objects with no linkage in non-type template arguments
- 6 **Polling Process**
- 7 **Teleconferences**
- 8 **Remaining Open Issues**
- 9 **Near-future EWG plans**

References

Informative References

§ 1. Executive summary

We have not met in-person since the February 2020 meeting in Prague because of the global pandemic. We're instead holding weekly teleconferences, as detailed in [\[P2145R1\]](#). We focus on providing non-final guidance, and will use electronic straw polls as detailed in [\[P2195R0\]](#) to move papers and issues forward in an asynchronous manner.

Our main achievements have been:

- **Issue processing:** most of the 50 language evolution issues have proposed resolutions, and a large number of them have been voted on through electronic polling.
- **C++23:** we've started work on papers for C++23 and later.
- **Incubation:** we've acted as EWG-I and "incubated" some early papers by providing early feedback to authors.

This paper outlines:

- The work achieved in [§ 7 Teleconferences](#);
- Lists the [§ 5 Poll](#) that were submitted for the May 2021 polling period and explain the [§ 6 Polling Process](#) (see [\[P1018r9\]](#) for the results of the February 2021 polling period);
- Results of those polls;
- Lists the remaining outstanding issues in [§ 8 Remaining Open Issues](#), some are still open while others are waiting for polling;
- [§ 9 Near-future EWG plans](#).

§ 2. Paper of note

- [\[P1000R4\]](#) C++ IS schedule
- [\[P0592R4\]](#) To boldly suggest an overall plan for C++23
- [\[P1999R0\]](#) Process: double-check evolutionary material via a Tentatively Ready status
- [\[P2195R0\]](#) Electronic Straw Polls
- [\[P2145R1\]](#) Evolving C++ Remotely

§ 3. Tentatively ready papers

Following our process in [\[P1999R0\]](#), we usually mark papers as tentatively ready for CWG. We would usually take a brief look at the next meeting, and if nothing particular concerns anyone, send them to CWG. However, given the pandemic, we've decided to provide guidance only in virtual teleconferences, and have an asynchronous polling mechanism to officially send papers to CWG or other groups as detailed in [\[P2195R0\]](#).

You can follow the lists of papers on GitHub:

- [tentatively ready papers](#),
- [EWG vote on me papers](#).

§ 4. Issue Processing

We've reviewed 50 Language Evolution issues at the Core groups' request, and have tentative resolutions for most. We don't want to poll all of these at the same time, and therefore only polled a subset in the February 2021 polling period, reserving other issues for later polling periods. We've will therefore only poll the "tentatively ready" issues (since they're tied to papers, and polled with said papers, as outlined below), as well as the "resolved" issues since telecon attendees believe that prior work has already addressed the issues.

[§ 8 Remaining Open Issues](#) contains a list of issues which aren't being voted on in this polling period.

§ 5. Poll

The polls which EWG will took in the May 2021 polling period are:

§ 5.1. P0847r6 Deducing `this`

- [\[P0847r6\]](#)
- [GitHub issue](#)
- **Highlight:** new mechanism for specifying or deducing the value category of the expression that a member-function is invoked on. In other words, a way to tell from within a member function whether the expression it's invoked on is an *lvalue* or an *rvalue*; whether it is `const` or `volatile`; and the expression's type. Non-`static` and non-`virtual` member functions can therefore use the syntax:

```
struct S { void memfun(this S const volatile && self); }; instead of
struct S { void memfun() const volatile && };.
```

This change has the following upsides:

- [It can greatly reduce code duplication](#);
 - [Enhances the CRTP pattern](#);
 - [Enables recursive lambdas](#);
 - [Makes it possible to have by-value member functions](#);
 - [Allows for SFINAE-friendly callables](#).
- 🗳️ **Poll:** Forward P0847r6 "Deducing `this`" to Core for C++23.

Poll votes:

SF	F	N	A	SA
17	13	2	1	0

Poll outcome: ✅ consensus.

Salient comments:

- This is a very attractive feature, but it is also a radical change, essentially doubling the ways member functions work. I would like to give this a little more bake time. — Against
- I support the general idea of the proposal but am skeptical of some of the sharp edges (particularly the distinction between "real" member functions, member functions which take an "object parameter," and static member functions). — Favor
- The facility removes the need to write CRTP, removes many cases of boilerplate, and gives us recursive lambdas. There are some remaining concerns about unforeseen consequences and integrations, so further implementation experience would be highly welcome before moving this all the way to a plenary poll. — Favor
- This is a very well-thought-out proposal that addresses several pain points in the language and I am eager to see it move forward. I do have reservations about the type of a pointer to an explicit object member function, however. In the paper, the authors argue convincingly that this new kind of member function is best thought of as a variation on non-static member functions, pointing out that this perspective simplifies the wording changes and also gives freedom to class authors to choose either implicit or explicit object member functions without impacting the class's interface. The decision to use ordinary function pointers instead of member-function pointers to refer to explicit object member functions undermines this goal; the decision to use an explicit object member function is no longer transparent to the user. I can understand why the authors made this decision, as there is no pointer-to-member-function type that allows for the `this` object to be passed by value. I'm not convinced that pass-by-value is worthwhile. I think a better trade-off here would be to require the `this` object to have a reference type. — Favor
- The paper is exhaustively written - and in my opinion the pros clearly outweigh the cons and the trade offs are worth it. — Strongly Favor
- Possibly the most important feature currently proposed for C++23. The benefits in terms of deduplicating code are motivation enough for the enhancement, but that's just the tip of the iceberg. — Strongly Favor

§ 5.2. P0849r7 `auto(x)`: `DECAY_COPY` in the language

- [\[P0849r7\]](#)
- [GitHub issue](#)
- **Highlight:** proposes `auto(x)` and `auto{x}` for casting `x` into a *prvalue* as if passing `x` as a function argument by value. The functionality appears as the *decay-copy* function in the standard for exposition only.
- 🗳️ **Poll:** Forward P0849r7 "auto(x): `DECAY_COPY` in the language" to Core for C++23, and affirm LEWG's guidance to forward it to Library.

Poll votes:

SF	F	N	A	SA
9	18	2	0	1

Poll outcome: ✅ consensus.

Salient comments:

- Nothing about this syntax is intuitive. Surely we can do better than sprinkle "auto" all over the language? — Strongly Against
- Yes. It does look a bit awkward, but this fills a curiously missing gap and the syntax is consistent after all. — Favor
- Once again, this is an expert feature that could technically be implemented by an obtuse library function. In this sense it is more syntax sugar than a newly invented language feature. But even then it makes the language more regular - a good thing in itself alone - and it expresses intent much better than possible alternatives. Compilers may take this as a syntactic marker to implement the semantic in a more optimized way. — Favor
- `DECAY_COPY` continues to become more important, and getting it into the language is useful. — Favor
- This seems to give a clear meaning to a "missing" section of the language syntax. — Strongly Favor
- This extension of the domain of `auto` is not only useful but fundamentally unsurprising: `auto` means the same thing as it does everywhere else, nor could it mean anything significantly different here. — Strongly Favor
- The situation where we deal with standard collections/algos and need to pass in an object too are fairly common. Some of those just work because the standard mandates it (and the implementations internally struggle with unaliasing), while in other cases it's burden for the programmer. And it's not a small one, with a huge penalty if missed. IME it's also easy to overlook on review. The workarounds are not nice and beyond the noise can leave unwanted stuff around. And source for additional bugs. The proposed `auto(x)` form fixes all that in an elegant way. That happens to handle both type correctness and avoid extra copy. While it leans on the "smart" side I don't share the concerns here, as it reflects the `T(x)` form and uses `auto` as many other places where `T` could be spelled out. And it's easy to teach. The considered alternatives are both more noisy and between "less intuitive" and "what does it mean" depending on the expert level. — Strongly Favor

§ 5.3. P2242r2 Non-literal variables (and labels and `gotos`) in `constexpr` functions

- [\[P2242r2\]](#)
- [GitHub issue](#)
- **Highlight:** strike the restriction that a `constexpr` function cannot contain a definition of a variable of non-literal type (or of `static` or thread storage duration), or a `goto` statement, or an identifier label. The rationale is briefly that the mere presence of the aforementioned things in a function is not in and of itself problematic; we can allow them to be present, as long as constant evaluation doesn't evaluate them.
- 🗳️ **Poll:** Forward P2242r2 "Non-literal variables (and labels and `gotos`) in `constexpr` functions" to Core for C++23.

Poll votes:

SF	F	N	A	SA
12	16	2	0	0

Poll outcome: ✅ consensus.

Salient comments:

- There's little not to like: it removes an unnecessary restriction on the non-constexpr code block without putting any new burden on the constexpr evaluator. — Strongly Favor
- This removes barriers to writing constexpr functions and continues the existing pattern of recognizing that potentially problematic constructs are only problematic if they're actually encountered during evaluation. — Strongly Favor

§ 5.4. CWG728 Restrictions on local classes

- [\[CWG728\]](#)

- **Issue:**

Now that the restriction against local classes being used as template arguments has been lifted, they are more useful, yet they are still crippled. For some reason or oversight, the restriction against local classes being templates or having member templates was not lifted. Allowing local classes to have member templates facilitates generic programming (the reason for lifting the other restriction), especially when it comes to the visitor-pattern (see the [boost::variant](#) documentation and the following example) as implemented in boost and the [boost::MPL](#) library (since functors have to be template classes in mpl, and higher-order functors have to have member templates to be useful). A local class with a member template would allow this desirable solution:

```
#include <boost/variant.hpp>
int main() {
    struct times_two_generic: public boost::static_visitor<> {
        template <typename T> void operator()(T& operand) const {
            operand += operand;
        }
    };
    std::vector<boost::variant<int, std::string>> vec;
    vec.push_back(21);
    vec.push_back("hello ");
    times_two_generic visitor;
    std::for_each(vec.begin(), vec.end(), boost::apply_visitor(visitor));
}
```

Is there any compelling reason not to allow this code? Is there any compelling reason not to allow local classes to be templates, have friends, or be able to define their static data members at function scope? Wouldn't this symmetry amongst local and non-local classes make the language more appealing and less embarrassing?

- Also see [\[P2044r2\]](#)[GitHub issue](#), seen by EWGI in Prague and requested to be merged with [\[P1988r1\]](#)[GitHub issue](#).
- **Meeting:** [2020-04-09 notes](#), some folks unsure that we should do anything here. Revisit the resolution with P2044. Suggest closing as Not A Defect because we have significant implementation concerns, but we'll explore the design space in P2044. All in favor.
- **Meeting:** [2020-09-24 notes](#) for the subsequent discussion of P2044, which expressed interest in fixing the issue but not necessarily in the proposed manner.
- 🗳️ **Poll:** Mark CWG728 "Restrictions on local classes" as Not A Defect.

Poll votes:

SF	F	N	A	SA
6	13	3	2	0

Poll outcome: ✅ consensus.

Salient comments:

- This is something I wish I could do, quite often. — Against
- The restriction seems unreasonable when we have more code that utilizes templates. — Against
- I submitted this issue - and i can understand the need for a paper. — Neutral

- The room felt like this may be an issue worth addressing, but it needs a paper. It's "not a defect" means there may still be a problem, but we should fix it with a feature change, not as a defect. I agree. — Favor
- I've written a paper about these restrictions because there are actual problems to solve in allowing these capabilities. It's not a defect. — Strongly Favor
- Certainly needs a proposal, not a good fit for a Core Issue. — Strongly Favor

§ 5.5. CWG944 `reinterpret_cast` for all types with the same size and alignment

- [\[CWG944\]](#)
- **Issue:** During the discussion of issue [\[CWG799\]](#), which specified the result of using `reinterpret_cast` to convert an operand to its own type, it was observed that it is probably reasonable to allow `reinterpret_cast` between any two types that have the same size and alignment. It has been suggested that this question may more properly be the province of EWG, especially in light of discussions during the resolution of issue [\[CWG330\]](#). CWG agreed that this question should be considered from a language design perspective and is thus being referred to EWG. Suggest rejection: we chose to add `std::bit_cast` to provide exactly this functionality instead of extending `reinterpret_cast`.
- **Meeting:** [2020-04-09 notes](#), suggest closing as Not A Defect. `bit_cast` addresses some of the usecases this issue might cover. We'd want separate papers to support other usecases, but have nothing outstanding at the moment. All in favor.
- 🗳️ **Poll:** Mark CWG944 "`reinterpret_cast` for all types with the same size and alignment" as Not A Defect.

Poll votes:

SF	F	N	A	SA
11	13	1	0	0

Poll outcome: ✅ consensus.

Salient comments:

- I'm less concerned with the syntax we use to express this (`reinterpret_cast`, `bit_cast`, etc., but I'd like to see more expressiveness here. — Neutral
- I agree that `std::bit_cast` exists to cover the key use cases. It also better conveys the idea of what the compiler is doing. — Strongly Favor
- `reinterpret_cast` is about object identity (whether as a glvalue, a pointer, or an integer holding a pointer value), not value, and it would be problematic for it to have two plausible meanings for an integer and a pointer that had the same size but were not interconvertible via `memcpy`. In other words, it's not `std::bit_cast`, and we have that now. — Strongly Favor

§ 5.6. CWG1451 Objects with no linkage in non-type template arguments

- [\[CWG1451\]](#)
- [GitHub issue](#)
- **Issue:**

According to 13.4.2 **[temp.arg.nontype]** paragraph 1 bullet 3, only objects with linkage can be used to form non-type template arguments. Is this restriction still needed? It would be convenient to use block-scope objects as template arguments. The following is valid:

```
template <int *> struct A;
auto f() {
    static int x;
    A<&x> *ap = nullptr;
    return ap;
}
```

There is no longer any linkage restriction. There is a restriction on storage duration, which seems reasonable (you can use the address of a local static variable but not of a local automatic variable as a non-type template argument).

- **Meeting:** [2020-04-15 notes](#), also [discussed in Rapperswil 2014](#), suggest closing as Not A Defect. Addressed by [\[N4268\]](#). No objection to unanimous consent.
- **Note:** we polled to close as "Not A Defect" but changed to poll these types of issues as "Resolved" in subsequent teleconferences, the poll therefore follows the more recent approach.
- **🗳️ Poll:** Mark CWG1451 "Objects with no linkage in non-type template arguments" as Resolved by N4268.

Poll votes:

SF	F	N	A	SA
10	14	0	0	0

Poll outcome:  consensus.

Salient comments:

- Seems clear this issue is resolved. — Strongly Favor

§ 6. Polling Process

For each poll, participants are asked to vote one of:

- Strongly in favor
- In favor
- Neutral
- Against
- Strongly against

Participants also have the option to abstain from voting on a particular poll. They are be asked to comment on each poll. This comment is mandatory, as it helps the chair determine consensus.

§ 7. Teleconferences

Here are the minutes for the virtual discussions that were held since the Prague meeting in February 2020:

1. 2020-04-09 [Issue Processing](#)
2. 2020-04-15 [Issue Processing](#)
3. 2020-04-23 [Issue Processing](#)
4. 2020-04-29 [Issue Processing](#)
5. 2020-05-07 [Issue Processing](#)
6. 2020-05-13 [Issue Processing](#)
7. 2020-05-21 [A General Property Customization Mechanism](#)—[\[P1393R0\]](#) ([P1393 tracking issue](#))
8. 2020-06-10 [Reviewing Deprecated Facilities of C++20 for C++23](#)—[\[P2139R1\]](#) ([P2139 tracking issue](#))
9. 2020-06-18 [C++ Identifier Syntax using Unicode Standard Annex 31](#)—[\[P1949R4\]](#) ([P1949 tracking issue](#))
10. 2020-06-24 [Extended floating-point types and standard names](#)—[\[P1467R4\]](#) ([P1467 tracking issue](#))
11. 2020-07-02 [Allow Duplicate Attributes, Attributes on Lambda-Expressions](#)—[\[P2156R0\]](#) ([P2156 tracking issue](#)) and [\[P2173R0\]](#) ([P2173 tracking issue](#))
12. 2020-07-08 [Pointer lifetime-end zap and provenance, too](#)—[\[P1726R3\]](#) ([P1726 tracking issue](#))
13. 2020-07-16 [Guaranteed copy elision for return variables](#)—[\[P2025R1\]](#) ([P2025 tracking issue](#))

14. 2020-07-30 [Reviewing Deprecated Facilities of C++20 for C++23, Removing Garbage Collection Support—](#)
[\[P2139R2\]](#) (P2139 tracking issue) and [\[P2186R0\]](#) (P2186 tracking issue)
15. 2020-08-05 [Transactional Memory Lite Support in C++—](#)[\[P1875R0\]](#) (P1875 tracking issue)
16. 2020-08-19 [Freestanding Language: Optional `::operator new`, Mixed string literal concatenation—](#)[\[P2013R2\]](#)
(P2013 tracking issue) and [\[P2201R0\]](#) (P2201 tracking issue)
17. 2020-08-27 [Exhaustiveness Checking for Pattern Matching—](#)[\[P1371R3\]](#) (P1371 tracking issue)
18. 2020-09-02 [#embed - a simple, scannable preprocessor-based resource acquisition method—](#)[\[P1967R2\]](#) (P1967
tracking issue)
19. 2020-09-10 [A pipeline-rewrite operator—](#)[\[P2011R1\]](#) (P2011 tracking issue)
20. 2020-09-16 [Pattern matching: inspect is always an expression—](#)[\[P1371R3\]](#) (P1371 tracking issue)
21. 2020-09-24 [C++ Identifier Syntax using Unicode Standard Annex 31, Member Templates for Local Classes—](#)
[\[P1949R6\]](#) (P1949 tracking issue) and [\[P2044R0\]](#) (P2044 tracking issue)
22. 2020-09-30 [Narrowing contextual conversions to bool, Generalized pack declaration and usage—](#)[\[P1401R3\]](#) (P1401
tracking issue) and [\[P1858R2\]](#) (P1858 tracking issue)
23. 2020-10-08 [Compound Literals, `if constexpr`—](#)[\[P2174R0\]](#) (P2174 tracking issue) and [\[P1938R1\]](#) (P1938 tracking
issue)
24. 2020-10-14 [Inline Namespaces: Fragility Bites, Trimming whitespaces before line splicing—](#)[\[P1701R1\]](#) (P1701
tracking issue) and [\[P2223R0\]](#) (P2223 tracking issue)
25. 2020-10-22 [Issues Processing](#)
26. 2020-10-28 [Issues Processing](#)
27. 2020-11-05 [Deducing `this`—](#)[P0847R5](#) (P0847 tracking issue)
28. 2020-11-19 [goto in pattern matching](#)
29. 2020-12-03 [auto\(x\): decay-copy in the language—](#)[P0849R5](#) (P0849 tracking issue)
30. 2021-01-14 [Polls](#)
31. 2021-01-20 [Final polls preparation](#)
32. 2021-01-28 [P2012 Fix the range-based for loop](#)
33. 2021-02-03 [P2280 Using unknown references in constant expressions](#)
34. 2021-02-11 [P1974 Non-transient constexpr allocation using `propconst`](#)
35. 2021-02-17 [P2242 Non-literal variables \(and labels and `goto`s\) in `constexpr` functions](#)
36. 2021-02-25 [P1371 pattern matching: implementation experience](#) pattern matching now has fairly capable prototype
implementation and in this session it will be both demonstrated and described. Bruno, the core implementer, will also
field any implementation related questions surrounding the proposal. See the [Godbolt demo](#).
37. 2021-03-02 [P2279R0 We need a language mechanism for customization points](#) joint EWG+LEWG session
38. 2021-03-11 [P2285 Are default function arguments in the immediate context?](#)
39. 2021-03-17 [P2266 Simpler implicit move](#)
40. 2021-03-25 [P2128 Multidimensional subscript operator](#)
41. 2021-03-31 [P2036 Changing scope for lambda trailing-return-type, and P2287 Designated-initializers for base classes](#)
42. 2021-04-08 [Pattern matching identifier patterns syntax: `let` vs `auto` vs `none`](#) Introducing bindings inside pattern
matching: Unqualified identifier within pattern matching must have well defined meaning. Published version of the
paper (P1371R4) proposed using case keyword to disambiguate names between identifier expressions and introducing
named binding. This direction raised some concerns within committee and authors want to propose another way of
resolving the ambiguity. After the presentation authors would like to have a poll to secure committee approval on the
proposed direction.
43. 2021-04-22 [P2334 Add support for preprocessing directives `elifdef` and `elifndef`, and P2246 Character encoding
of diagnostic text](#)
44. 2021-04-28 [Final discussion of the polls, and P2066 Suggested draft TS for C++ Extensions for Minimal Transactional
Memory](#)

45. 2021-05-06 [P2314 Character sets and encodings, and quick review of r2 for P2280 Using unknown references in constant expressions](#)
46. 2021-05-12 [P2255 A type trait to detect reference binding to temporary](#)
47. 2021-05-20 [P2025 Guaranteed copy elision for named return objects](#)
48. 2021-05-26 [P2041 Deleting variable templates](#)
49. 2021-06-03 [P1967 #embed — a simple, scannable preprocessor-based resource acquisition method](#)

§ 8. Remaining Open Issues

The following table lists all remaining open issues referred to EWG by Core or Library. Some of them are ready to be polled but are held back from the polling periods to limit the number of polls in this round.

From	#	Title	Notes
Core	[CWG2261]	Explicit instantiation of in-class friend definition	<pre>struct S { template <class T> friend void f(T) { } }; template void f(int); // Well-formed?</pre> A friend is not found by ordinary name lookup until it is explicitly declared in the containing name but declaration matching does not use ordinary name lookup. There is implementation divergence on handling of this example. Note 2020-04-29 Tentative agreement: This should be well-formed. SF 1 F 10 N 2 A 1 SA 0 JF emailed EWG / Core about this. Davis: the current name lookup approach which Core is taking in p1787 would disallow this. Support is possible, it would be inconsistent, but would also be a feature. Notes 2020-10-22: wait until p1787 is voted into the working draft, because it's making this behavior intentional. At that point, we can vote on marking the issue as Not a Defect. No objection to unanimous consent.
Core	[CWG2270]	Non- inline functions and explicit instantiation declarations	[Detailed description pending.] Hubert: question over the role of the inline keyword in relation to explicit instantiation declaration For inline functions, explicit instantiation declarations do not have the effect of suppressing implicit instantiation. A user's desire for wanting to suppress implicit instantiation can arise for different reasons: To reduce space in object files, executables, etc. and similarly to reduce the number of input symbols linker To reduce compile time in performing semantic analysis for instantiations whose definitions are provided elsewhere To control point-of-instantiation, avoiding contexts where the requisite declarations are not declared The special rule around inline functions allows inline-ness to be used to indicate that the first reasc intent and that instantiation-for-inlining is okay.

			Consider the following as a translation unit: <pre> template <typename T> //inline void f(T t) { g(t); } enum E : int; extern template void f(E); void h(E e) { f(e); } </pre> Marking the template definition <code>inline</code> would mean that the intended declaration for <code>g</code> would need provided as the best candidate at the points-of-instantiation for <code>f<E></code>. The issue initially points out that this use of the <code>inline</code> keyword does not match a view that <code>inline</code> essentially an ODR tool to allow multiple definitions (as opposed to a way to indicate desire for inlining). The issue proposed that <code>extern template</code> merely has the effect of suppressing definitions in terms of linkage (regardless of the <code>inline</code> keyword). Such a change would affect the usability of the feature for user that falls within the latter two options above. I am not sure if CWG is asking EWG a specific question other than the general "we do not believe this wording or obvious consistency issue; is this an issue in terms of design?" Meeting: Hubert forked the thread on the reflector. Might want the education SG to take a look, or might want Meeting 2020-10-28: Inbal will try to put together the wording, to prove / disprove whether this is a C
Lib	[LWG2432]	initializer_list assignability	std::initializer_list::operator= [support.initlist] is horribly broken and it needs deprecation: <pre> std::initializer_list<foo> a = {{1}, {2}, {3}}; a = {{4}, {5}, {6}}; // New sequence is already destroyed. </pre> Assignability of <code>initializer_list</code> isn't explicitly specified, but most implementations supply a default assignment operator. I'm not sure what [description] says, but it probably doesn't matter. Proposed resolution: Edit [support.initlist] p1, class template <code>initializer_list</code> synopsis, as indicated: <pre> namespace std { template<class E> class initializer_list { public: [...] constexpr initializer_list() noexcept; initializer_list(const initializer_list&) = default; initializer_list(initializer_list&&) = default; </pre>

			<pre>initializer_list& operator=(const initializer_list&) = delete; initializer_list& operator=(initializer_list&&) = delete; constexpr size_t size() const noexcept; [...] }; [...] }</pre> LWG telecon appears to want a language change to disallow assigning a braced-init-list to an <code>std::initializer_list</code> but still permit move assignment of <code>std::initializer_list</code> objects. That is, <pre>auto il1 = {1,2,3}; auto il2 = {4,5,6}; il1 = {7,8,9}; // currently well-formed but dangles immediately; should be ill-formed il1 = std::move(il2); // currently well-formed and should remain so</pre> Meeting: Proposed resolution: <pre>initializer_list(const initializer_list&) = default; initializer_list(initializer_list&&) = default; [[deprecated]] initializer_list& operator=(const initializer_list&) = default; [[deprecated]] initializer_list& operator=(initializer_list&&) = default;</pre> SF F N A SA 0 3 12 0 0 JF emailed LEWG, to see if they have an opinion, no feedback. Asked LEWG chairs to schedule for a telecon. LWG discussed priority.
Lib	[LWG2813]	<code>std::function</code> should not return dangling references	If a <code>std::function</code> has a reference as a return type, and that reference binds to a prvalue returned by the function, then the reference is always dangling. Because any use of such a reference results in undefined behaviour, the <code>std::function</code> should not be allowed to be initialized with such a callable. Instead, the program should be ill-formed. A minimal example of well-formed code under the current standard that exhibits this issue: <pre>int main() { std::function<const int&()> F([]{ return 42; }); int x = F(); // oops! }</pre>

			Proposed resolution: Add a second paragraph to the remarks section of 20.14.16.2.1 [func.wrap.func.con]: template<class F> function(F f); -7- Requires: F shall be CopyConstructible. -8- Remarks: This constructor template shall not participate in overload resolution unless • F is Lvalue-Callable (20.14.16.2 [func.wrap.func]) for argument types ArgTypes... and return type U and • If R is type "reference to T" and INVOKE(ArgTypes...) has value category V and type U: ◦ V is a prvalue, U is a class type, and T is not reference-related (9.4.3 [dcl.init.ref]) to U, and ◦ V is an lvalue or xvalue, and either U is a class type or T is reference-related to U. [...] Tim: LWG in Batavia 2018 would like a way to detect when the initialization of a reference would be a temporary. This requires compiler support, since there's no known way in the current language to do this reliably in the presence of user-defined conversions (see thread starting at https://lists.isocpp.org/lib/2017/07/3256.php). Meeting: Tim wrote p2255 to address this. Ville thinks there should be an analysis of alternative approaches. Also see P0932.
Core	[CWG794]	Base-derived conversion in member type of pointer-to-member conversion	Related to CWG 170, drafting by Clark seems unlikely. This is section 2.1 of Jeff Snyder's P0149R0, which was approved by EWG, 4 years ago, waiting for wording. JF reached out to Jeff. Did wording with Jens, main blocker is lack of implementation. Meeting: (notes) Suggest closing as Not A Defect because we have implementation uncertainties, but explore the design space in P0149. ABI group will discuss. All in favor.
Core	[CWG900]	Lifetime of temporaries in range-based for	// some function <pre>std::vector<int> foo();</pre> // correct usage <pre>auto v = foo();</pre> <pre>for(auto i : reverse(v)) { std::cout << i << std::endl; }</pre> // problematic usage <pre>for(auto i : reverse(foo())) { std::cout << i << std::endl; }</pre> Meeting: (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect because it's a change which might have effects on existing code (might cause bugs), and might need to change more than just the range-based for loop.

			range-based loops. See p0614, p0577, p0936, p1179. We'll explore the design space in a separate paper circled back with Nico on this, might write a paper. All in favor.
Core	[CWG1008]	Querying the alignment of an object	https://godbolt.org/z/_SN8iF • GCC already implements this extension without issuing a warning • Clang and EDG implement this extension for gcc compatibility, with a warning • MSVC does not yet implement this feature Quick example using 'auto' illustrates why we might want this capability for objects as well as types. Principle of least astonishment suggests it is surprising for sizeof and alignof to behave differently in regard. Recommend shipping this straight to core as soon as we can find a wording champion. We need to discuss with WG14. Questions for EWG to answer before forwarding: • Should this be a unary operator, like sizeof, so "alignof x" is valid? Or should it be like typeid, where parens are required? • What would this mean? Is it the alignment of the type of the object? Or the compiler's best guess: alignment of the expression itself? Should it take into account any facts that are known about the expression other than its type? If so, which ones? (For example, if applied to an expression x or x is declared with an alignas attribute, is that value returned?) This needs a design paper rather than going straight to core. https://godbolt.org/z/TeVA9T GCC seems to report the alignment of the object not just of decltype(object) Meeting: (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect, the design is especially around alignment of object versus type. Invite a paper, Inbal will pitch in, Alidair can collate. All in favor. Inbal's paper: P2152R0
Core	[CWG1077]	Explicit specializations in non-containing namespaces	The current wording of 9.8.1.2 [namespace.memdef] and 13.9.3 [temp.expl.spec] requires that an explicit specialization be declared either in the same namespace as the template or in an enclosing namespace. It would be convenient to relax that requirement and allow the specialization to be declared in a non-enclosing namespace to which one or more of the template arguments belongs. Additional note, April, 2015: See EWG issue 48. Might allow us to revert DR2061 and all the horribleness that created and described in p1701. The problem p1077 addresses is the motivating factor in dr2061. Also see CWG 2370. Meeting: (notes) Suggest closing as Not A Defect. See p0665, minutes. Continue under p0665 or a future version of it. All in favor.
Core	[CWG1433]	trailing-return-type and point of declaration	<pre>template <class T> T list(T x); template <class H, class ...T> auto list(H h, T ...args) -> decltype(list(args...)); // list isn't in scope in its own trailing-return-type auto list3 = list(1, 2, 3);</pre> Meeting: (notes, also discussed in Rapperswil 2014) there might be compiler divergence according to Daveed. Suggest closing as Not A Defect, it would be tricky to change behavior without ambiguity. "I

			this would break existing code that relies on seeing only previous declarations. No objection to unanimous consent.
Core	[CWG1469]	Omitted bound in array <i>new-expression</i>	The syntax for noptr-new-declarator in 7.6.2.7 [expr.new] paragraph 1 requires an expression, even though the bound could be inferred from a braced-init-list initializer. It is not clear whether 9.4.1 [dcl.init.aggr] paragraph 4, An array of unknown size initialized with a brace-enclosed initializer-list containing n initializers, where n shall be greater than zero, is defined as having n elements (9.3.3.4 [dcl.array]). should be considered to apply to the new-type-id variant, e.g., <pre>new (int[]){1, 2, 3}</pre> This was addressed by p1009 Meeting: (notes) Suggest closing as Not A Defect. Addressed by P1009. No objection to unanimous consent.
Core	[CWG1555]	Language linkage and function type compatibility	Currently function types with different language linkage are not compatible, and 7.6.1.2 [expr.call] paragraph 1 makes it undefined behavior to call a function via a type with a different language linkage. These restrictions are generally not enforced by most current implementations (although some do) between functions with C and C++ language linkage. Should these restrictions be relaxed, perhaps as conditionally-supported by the implementation? Somewhat related to CWG1463. Meeting: (notes) no strong consensus at the moment, Erich Keane brought this up with SG12 Undefined Behavior. Long discussion, will need to revisit, need a paper. Meeting Oct 22nd 2020: will need a volunteer to write a paper, but the wording in the standard is unambiguous, and we have existence proof of platforms which use different calling conventions between C and C++. However many major compilers have chosen to ignore this. Not a defect, but we welcome a proposal to change the status quo. No objection to unanimous consent.
Core	[CWG1643]	Default arguments for template parameter packs	Although 13.2 [temp.param] paragraph 9 forbids default arguments for template parameter packs, all compilers would make some program patterns easier to write. Should this restriction be removed? Meeting: (notes) Suggest closing as Not A Defect. Interesting design space, but needs a paper, see N1756. No objection to unanimous consent.
Core	[CWG1864]	List-initialization of array objects	The resolution of issue 1467 now allows for initialization of aggregate classes from an object of the same type. Similar treatment should be afforded to array aggregates. See recent discussion on allowing 'auto x[] = {1, 2, 3};' -- this topic came up there. The two questions were answered independently, but some consider them to be related. Meeting: (notes) Suggest closing as Not A Defect. Fixing anything in this space would require a paper that considers the entire design space, Timur might be interested in this. No objection to unanimous consent.
Core	[CWG1871]	Non-identifier characters in <i>ud-suffix</i>	JF forwarded to SG16 Unicode given their work on TR31. Tracked on GitHub. SG16 reviewed D194 and still needs wording changes. Discussed at the April 22nd SG16 telecon. SG16 Poll: Is there any objection to unanimous consent for recommending rejection of this proposal? No objection to unanimous consent. EWG Meeting: (notes 2020-04-15, notes 2020-05-07) Suggest closing as Not A Defect. No objection to unanimous consent.
Core	[CWG1876]	Preventing explicit specialization	A desire has been expressed for a mechanism to prevent explicitly specializing a given class template particular std::initializer_list and perhaps some others in the standard library. It is not clear whether simply adding a prohibition to the description of the templates in the library clauses would be sufficient or whether a core language mechanism is required.

			Meeting: (notes) Suggest closing as Not A Defect. No objection to unanimous consent. This could be language feature, would need library usecase examples. Poll: we are interested in such a paper SF 2 F 13 N 6 A 1 SA 0
Core	[CWG1915]	Potentially-invoked destructors in non-throwing constructors	Since the base class constructor is non-throwing, the deleted base class destructor need not be referenced. There's a typo in the issues list here: it should say "Since the <i>derived</i> class constructor is non-throwing, the deleted base class destructor need not be referenced." Meeting: (notes 2020-04-23) the proposed wording changes the implementation leeway in two-phase unwinding, which breaks some existing ABIs. We would need a paper to explore this further. Suggest closing as Not A Defect. No objection to unanimous consent.
Core	[CWG1923]	Lvalues of type void	There does not seem to be any significant technical obstacle to allowing a void* pointer to be dereferenced and that would avoid having to use heavyweight circumlocutions when casting to a reference to an object designated by such a pointer. Might consider this a duplicate of the discussion on "regular void". JF reached out to Matt. He has an implementation in clang. Needs to update the paper. Might need a v to present. Daveed would be interested in presenting. Meeting: (notes 2020-04-23) Suggest closing as Not A Defect. Explore under the "regular void" umbrella. No objection to unanimous consent.
Core	[CWG1934]	Relaxing exception-specification compatibility requirements	According to 14.5 [except.spec] paragraph 4, If any declaration of a function has an exception-specification that is not a noexcept-specification, all declarations, including the definition and any explicit specialization, of that function shall have a compatible exception-specification. This seems excessive for explicit specializations, considering that paragraph 6 applies a looser requirement for virtual functions: If a virtual function has an exception-specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception-specification of the base class virtual function. The rule in paragraph 3 is also problematic in regard to explicit specializations of destructors and default special member functions, as the implicit exception-specification of the template member function cannot be determined. There is also a related problem with defaulted special member functions and exception-specifications. According to 9.5.2 [dcl.fct.def.default] paragraph 3, If a function that is explicitly defaulted has an explicit exception-specification that is not compatible ([except.spec]) with the exception-specification on the implicit declaration, then • if the function is explicitly defaulted on its first declaration, it is defined as deleted; • otherwise, the program is ill-formed.

			This rule precludes defaulting a virtual base class destructor or copy/move functions if the derived class function will throw an exception not allowed by the implicit base class member function. Meeting: (notes 2020-04-23) JF will work with Mike to update wording to C++20. Will revisit. From Mike: This issue is NAD since we eliminated typed exception-specifications. The current word dealing only with noexcept(true) and noexcept(false), does not have this issue. Will remove “extension status.” Meeting: no objection to CWG taking it back, and marking it as NAD.
Core	[CWG1957]	decltype(auto) with direct-list-initialization	Paper N3922 changed the rules for deduction from a braced-init-list containing a single expression in initialization context. Should a corresponding change be made for decltype(auto)? E.g., <pre>auto x8a = { 1 }; // decltype(x8a) is std::initializer_list<int></pre> decltype(auto) x8d = { 1 }; // ill-formed, a braced-init-list is not an expression <pre>auto x9a{ 1 }; // decltype(x9a) is int</pre> decltype(auto) x9d{ 1 }; // decltype(x9d) is int See also issue 1467, which also effectively ignores braces around a single expression, this change was parallel to that one, even though the primary motivation for decltype(auto) is in the return type of a forwarding function, where direct-initialization does not apply. Meeting: (notes 2020-04-23) Suggest closing as Not A Defect. This would be a language change, it's that we want to make such a change. It would require a paper. Mike Spertus is writing a paper with some overlap, will cover this as well. No objection to unanimous consent.
Core	[CWG2111]	Array temporaries in reference binding	The current wording of the Standard appears to permit code like <pre>void f(const char (&)[10]); void g() { f("123"); f({'a', 'b', 'c', '\0'}); }</pre> creating a temporary array of ten elements and binding the parameter reference to it. This is controversial and should be reconsidered. (See issues 1058 and 1232.) Meeting: (notes 2020-04-23) JF digging more, talking to Richard about this. Somewhat related to P2174 compound literals. Would be very strange if <pre>const char (&amp;x)[10] = {'a', 'b', 'c'};</pre> ... were invalid but ... <pre>const char x[10] = {'a', 'b', 'c'};</pre> <pre>const char (&amp;x)[3] = {'a', 'b', 'c'};</pre> ... were both OK. Maybe either we should allow the trailing elements to be zeroed in general (the status quo) or not (a major breaking change). Which means we should reject the issue on consistency grounds. The general rule is that if

			T &&r = init; ... can't bind directly, we create a temporary initialized as if with T r = init; ... and bind to that. (And similarly for const references.) Another question: Do we want to support (T){inits} as a synonym for T{inits}? Meeting Oct 22nd 2020: Note that the issue itself is defective, and <code>f("123")</code> isn't valid. CWG2352 references p1358, and was resolved, leading us to believe that this is now mainstream and controversial. Not a defect. No objection to unanimous consent.
Core	[CWG2125]	Copy elision and comma operator	Currently, _N4750_.15.8 [class.copy] paragraphs 31-32 apply only to the name of a local variable in determining whether a return expression is a candidate for copy elision or move construction. Would sense to extend that to include the right operand of a comma operator? <pre>X f() { X x; return (0, x); }</pre> Meeting: (notes 2020-04-23) Consider expanding to other places that expand bit-field-ness such as <code>throw : x;</code>. Suggest closing as Not A Defect. Will need a paper to address, no current volunteer for this objection to unanimous consent.
Core	[CWG2132]	Deprecated default generated copy constructors	EWG has indicated that they are not currently in favor of removing the implicitly declared defaulted copy constructors and assignment operators that are deprecated in _N4750_.15.8 [class.copy] paragraphs 7 & 8. Should this deprecation be removed? Related: discussing under p2139 deprecations. Meeting: (note 2020-04-29) Suggest closing as Not A Defect. No objection to unanimous consent. We want to remove entirely, or un-deprecate. This will need a paper, Ville will talk with Alisdair about for the topic from p2139.
Core	[CWG914]	Value-initialization of array types	Although value-initialization is defined for array types and the {} initializer is permitted in a mem-init naming an array member of a class, the syntax T{} (where T is an array type) is explicitly forbidden by [expr.type.conv] paragraph 2. This is inconsistent and the syntax should be permitted. Rationale (July, 2009): The CWG was not convinced of the utility of this extension, especially in light of questions about handling the lifetime of temporary arrays. This suggestion needs a proposal and analysis before it can be considered by the CWG. This has become a more severe inconsistency after we adopted Ville's P0960 for C++20. Now it's not just T{} that has this weird special-case restriction, it's (a, b, c) too: <pre>using X = int[]; X x{1, 2, 3}; // ok, int[3] X y(1, 2, 3); // ok, int[3] f(X{1, 2, 3}); // ok, int[3] temporary</pre>

			f(X(1, 2, 3)); // error Meeting: (notes) it is a defect, need a paper. David Stone trying to find a volunteer to write said paper favor.
Core	[CWG1463]	extern "C" alias templates	Currently 13 [temp] paragraph 4 forbids any template from having C linkage. Should alias templates exempt from this prohibition, since they do not have any linkage? Additional note, April, 2013: It was suggested (see messages 23364 through 23367) that relaxing this restriction for alias templates could provide a way of addressing the long-standing lack of a way of specifying a language linkage for a dependent function type (see issue 13). The priority was deleted to allow CWG consider the implications of that potential application of the facility. We should either have some way to express a dependent function type with C language linkage (and accept 1555 below) or we should remove the notion that C language linkage (or not) is part of the function type at all. (Apparently some targets used it; are they still in use? EDG probably knows.) Actively discussed on CWG reflector. Davis' interpretation of CWG discussion: I don't speak for Core, of course, but my (Evolutionary) thoughts are those given (last) in the message which you replied: neither the wording nor the apparent intent of [temp.pre]/6's restrictions on C linkage of templates make any sense. Since templates (and explicit specializations) can't have C linkage of the mangling variety (which the standard calls language linkage of a (function) name) but can have C linkage of the calling-convention variety (which the standard calls language linkage of a (function) type), extern should grant them the latter and not the former with no error. This has certain obvious applications in C APIs with callbacks. (Put differently, CWG13 shouldn't have been rejected; it appears to have gotten bogged down in questions of syntax, but we have an adequate syntactic workaround that we merely haven't permit.) CWG1463 asks for a (very) proper subset of the above: merely that extern "C" be allowed to apply to templates (claiming incorrectly that they have no name with linkage at all). I consider it more relevant and more productive to talk about whether entities have names with language linkage than with the ordinary kind. The Core reflector discussion (which seems to have finished for the moment) also touched on the case where "the same" function template is declared with two different language linkages for its type, but the only relevant evolution input there would be a decision to go the opposite way and forbid function templates from having types with C language linkage entirely. (They currently can, but it's mostly or entirely useless.) If (for CWG1555, also on your list) the rules about language linkage of (function) types are sufficiently relaxed, then CWG1463 may be moot (and my extension of it with it), but that seems unlikely given the recent identification of a case where they matter. Meeting: (notes 2020-04-15, notes 2020-10-22, also discussed in Rapperswil 2014) We agree that this issue. extern "C" on a template should only affect calling convention, and not the mangling. Davis has a Hubert volunteer to write a paper. Unanimous consent.
Core	[CWG1790]	Ellipsis following function parameter pack	Although the current wording permits an ellipsis to immediately follow a function parameter pack, it is clear that the <csdarg> facilities permit access to the ellipsis arguments. The problem here (which is not explained in the issue) is: how do you supply the name of the last parameter before the ellipsis to va_start? You can't put the name of a pack there (it wouldn't be expanded) and there is no way to name the last element of the pack (nor to deal with the case where the pack is empty). Meeting: (notes) 3 options: fix wording around "last parameter", remove facility entirely (either va_start or function declarator), try to invent a language facility. JF emailed EWG to see if anyone has a strong

			preference, or if we should send back to CWG to fix wording, long discussion. Michael Spertus: I am willing to commit to including and analysis on this in an upcoming paper on parameter packs. JF followed up with Michael and Barry, no response.
Core	[CWG1962]	Type of __func__	Two questions have arisen regarding the treatment of the type of the __func__ built-in variable. First, implementations accept <pre>void f() { typedef decltype(__func__) T; T x = __func__; }</pre> even though T is specified to be an array type. In a related question, it was noted that __func__ is implicitly required to be unique in each function, and not only the value but the type of __func__ are implementation-defined; e.g., in something like <pre>inline auto f() { return &__func__; }</pre> the function type is implementation-specific. These concerns could be addressed by making the value prvalue of type const char* instead of an array lvalue. Rationale (November, 2018): See also issue 2362, which asks for the ability to use __func__ in a constexpr function. These two goals are incompatible. The deep question here is about __func__ and the ODR. Does EWG want implementations to somehow behave as if __func__ is the same in all copies of an inline function (in which case it can have an address and be usable in constant expressions, but the <i>demangling</i> algorithm used to construct it becomes problematic for ABI), or does EWG want implementations to behave as if __func__ may differ between copies, so its address is not known until runtime (in which case it must have either pointer or incomplete array type, and its value is not usable in constant expressions -- but its address could still be usable). Note: C++20 has std::source_location::function_name. Meeting: (notes 2020-04-23) We should discuss this with WG14. This is indeed a language issue. No objection to unanimous consent. We'll need a paper, potentially considering what Reflection could do brought it up on the mailing list. We probably need a paper to disentangle this.
Core	[CWG2228]	Ambiguity resolution for cast to function type	Proposed resolution. C++ has a blanket disambiguation rule that says if a sequence of tokens can be interpreted as either a name or an expression, the type-name interpretation is chosen. This is unhelpful in cases like <pre>(T())+3</pre> where the current rules make "(T())" a cast to the type "function with no parameters returning T" rather than parenthesized value-initialization of a temporary of type T. The former interpretation is always ill-formed because you can't cast to a function type - while the latter could be useful.

			Richard’s proposed resolution, cited above, is to avoid the ambiguity by changing the grammar so that like "(T())" cannot be parsed as a cast. The wording in the proposal applies that change to a number of different contexts where the ambiguity can come into play. There are two contexts where the change is applied, however: 1) the operand of typeid, and 2) a template-argument. During our discussion yesterday there was some support for the idea of applying the change to typeid, as well, although that would be a breaking change for any programs that rely on the current disambiguation to get type information for function types. There was general agreement, however, to exclude template arguments, since they are for things like std::function. CWG would, therefore, like some guidance from EWG on two questions. First, should we apply the new syntax to the operand of typeid, even though it’s a breaking change? More generally, the question was raised whether we should make this change at all. Although resolving the ambiguity in the other direction would arguably be more convenient in many cases, there is a tension between convenience and the complexity of the language. In particular, we would be creating a situation where the exact same sequence of tokens would mean two different things, depending on the context in which they appear. Is the convenience worth the cost in complexity? Meeting: (note 2020-04-29, notes 2020-03-23, note from Core summer 2020, notes from Core Prague 2019-01-07, notes from Core Kona 2019, notes from Core Cologne 2019) This is an open question we’d like to change the standard to resolve it: SF 0 F 6 N 4 A 3 SA 2 Davis emailed EWG reflector. Long discussion.
Core	[CWG2296]	Are default argument instantiation failures in the “immediate context”?	Example 1: <pre>template <typename T, typename U = T> void fun(T v, U u = U()) {} void fun(...) {} struct X { X(int) {} // no default ctor }; int main() { fun (X(1)); }</pre> Consider the following example (taken from issue 3 of paper P0348R0): Example 2: <pre>template <typename U> void fun(U u = U()); struct X { X(int) {} }; template <class T> decltype(fun<T>()) g(int) { } template <class> void g(long) { }</pre>

			<pre>int main() { g<X>(0); }</pre> When is the substitution into the return type done? The current specification makes this example ill-formed because the failure to instantiate the default argument in the decltype operand is not in the immediate context of the substitution, although a plausible argument for making this a SFINAE case can be made. Meeting: (notes 2020-05-07) The first example under issue 3 of paper P0348R0 should become well-formed. <pre>SF F N A SA 0 1 4 3 6</pre> The second example under issue 3 of paper P0348R0 should become well-formed. <pre>SF F N A SA 1 5 7 3 0</pre> This is an issue. We'd like to see a paper addressing it. It should explore what "Immediate context" means. No objection to unanimous consent. Daveed / Hubert might entertain writing a paper explaining this. Ville emailed EWG about this. JF contacted Andrzej to see if he's interested in addressing this. Not sure he is. Meeting 2020-10-28: Tomasz will write a paper which exposes the issue and what he thinks should be done (but not resolving wording itself). Hubert remembers an email about this, will find it and sync with JF. Meeting 2021-01-14: Andrzej wrote a paper for this (emailed 2021-01-12 to EWG).
Core	[CWG2362]	__func__ should be constexpr	The definition of __func__ in 9.5.1 [dcl.fct.def.general] paragraph 8 is: <pre>static const char __func__[] = "function-name";</pre> This prohibits its use in constant expressions, e.g., <pre>int main () { // error: the value of __func__ is not usable in a constant expression constexpr char c = __func__[0]; }</pre> Rationale (November, 2018): See also issue 1962, which asks that the type of __func__ be const char*. The two goals are incompatible. Meeting: handle with 1962.

§ 9. Near-future EWG plans

We will continue to work on issue resolution and C++23, prioritizing according to [\[P0592R4\]](#).

§ References

§ Informative References

[CWG1008]

Steve Clamage. [Querying the alignment of an object](https://wg21.link/cwg1008). 27 November 2009. extension. URL: <https://wg21.link/cwg1008>

[CWG1077]

Mike Spertus. [Explicit specializations in non-containing namespaces](https://wg21.link/cwg1077). 13 June 2010. extension. URL: <https://wg21.link/cwg1077>

[CWG1433]

Jason Merrill. [trailing-return-type and point of declaration](https://wg21.link/cwg1433). 20 December 2011. extension. URL: <https://wg21.link/cwg1433>

[CWG1451]

Daniel Krüger. [Objects with no linkage in non-type template arguments](https://wg21.link/cwg1451). 1 February 2012. CD4. URL: <https://wg21.link/cwg1451>

[CWG1463]

Daveed Vandevoorde. [extern "C" alias templates](https://wg21.link/cwg1463). 19 August 2011. extension. URL: <https://wg21.link/cwg1463>

[CWG1469]

Johannes Schaub. [Omitted bound in array new-expression](https://wg21.link/cwg1469). 12 February 2012. extension. URL: <https://wg21.link/cwg1469>

[CWG1555]

Daniel Krüger. [Language linkage and function type compatibility](https://wg21.link/cwg1555). 18 September 2012. extension. URL: <https://wg21.link/cwg1555>

[CWG1643]

Vinny Romano. [Default arguments for template parameter packs](https://wg21.link/cwg1643). 17 March 2013. extension. URL: <https://wg21.link/cwg1643>

[CWG1790]

Daryle Walker. [Ellipsis following function parameter pack](https://wg21.link/cwg1790). 1 October 2013. extension. URL: <https://wg21.link/cwg1790>

[CWG1864]

Alisdair Meredith. [List-initialization of array objects](https://wg21.link/cwg1864). 15 February 2014. extension. URL: <https://wg21.link/cwg1864>

[CWG1871]

Richard Smith. [Non-identifier characters in ud-suffix](https://wg21.link/cwg1871). 17 February 2014. extension. URL: <https://wg21.link/cwg1871>

[CWG1876]

John Spicer. [Preventing explicit specialization](https://wg21.link/cwg1876). 19 February 2014. extension. URL: <https://wg21.link/cwg1876>

[CWG1915]

Aaron Ballman. [Potentially-invoked destructors in non-throwing constructors](https://wg21.link/cwg1915). 15 April 2014. extension. URL: <https://wg21.link/cwg1915>

[CWG1923]

Richard Smith. [Lvalues of type void](https://wg21.link/cwg1923). 6 May 2014. extension. URL: <https://wg21.link/cwg1923>

[CWG1934]

Vinny Romano. [Relaxing exception-specification compatibility requirements](https://wg21.link/cwg1934). 3 June 2014. extension. URL: <https://wg21.link/cwg1934>

[CWG1957]

Vinny Romano. [decltype\(auto\) with direct-list-initialization](https://wg21.link/cwg1957). 20140630. extension. URL: <https://wg21.link/cwg1957>

[CWG1962]

Steve Clamage. [Type of __func__](https://wg21.link/cwg1962). 4 July 2014. extension. URL: <https://wg21.link/cwg1962>

[CWG2111]

Vinny Romano. [Array temporaries in reference binding](https://wg21.link/cwg2111). 2 April 2015. extension. URL: <https://wg21.link/cwg2111>

[CWG2125]

Vinny Romano. [Copy elision and comma operator](https://wg21.link/cwg2125). 6 May 2015. extension. URL: <https://wg21.link/cwg2125>

[CWG2132]

Nico Josuttis. [Deprecated default generated copy constructors](https://wg21.link/cwg2132). 28 May 2015. extension. URL: <https://wg21.link/cwg2132>

- [CWG2228]
Richard Smith. [Ambiguity resolution for cast to function type](https://wg21.link/cwg2228). 2 February 2016. review. URL: <https://wg21.link/cwg2228>
- [CWG2261]
Jens Maurer. [Explicit instantiation of in-class friend definition](https://wg21.link/cwg2261). 28 April 2016. extension. URL: <https://wg21.link/cwg2261>
- [CWG2270]
Richard Smith. [Non-inline functions and explicit instantiation declarations](https://wg21.link/cwg2270). 10 June 2016. extension. URL: <https://wg21.link/cwg2270>
- [CWG2296]
Jason Merrill. [Are default argument instantiation failures in the “immediate context”?](https://wg21.link/cwg2296). 25 June 2016. extension. URL: <https://wg21.link/cwg2296>
- [CWG2362]
Anthony Polukhin. [__func__ should be constexpr](https://wg21.link/cwg2362). 23 October 2017. extension. URL: <https://wg21.link/cwg2362>
- [CWG330]
Roger Orr. [Qualification conversions and pointers to arrays of pointers](https://wg21.link/cwg330). 2 January 2002. CD4. URL: <https://wg21.link/cwg330>
- [CWG728]
Faisal Vali. [Restrictions on local classes](https://wg21.link/cwg728). 5 October 2008. extension. URL: <https://wg21.link/cwg728>
- [CWG794]
CH. [Base-derived conversion in member type of pointer-to-member conversion](https://wg21.link/cwg794). 3 March 2009. extension. URL: <https://wg21.link/cwg794>
- [CWG799]
UK. [Can reinterpret_cast be used to cast an operand to its own type?](https://wg21.link/cwg799). 3 March 2009. CD2. URL: <https://wg21.link/cwg799>
- [CWG900]
Thomas J. Gritzan. [Lifetime of temporaries in range-based for](https://wg21.link/cwg900). 12 May 2009. extension. URL: <https://wg21.link/cwg900>
- [CWG914]
Gabriel Dos Reis. [Value-initialization of array types](https://wg21.link/cwg914). 10 June 2009. extension. URL: <https://wg21.link/cwg914>
- [CWG944]
Miller. [reinterpret_cast for all types with the same size and alignment](https://wg21.link/cwg944). 15 July 2009. extension. URL: <https://wg21.link/cwg944>
- [LWG2432]
David Krauss. [initializer_list assignability](https://wg21.link/lwg2432). EWG. URL: <https://wg21.link/lwg2432>
- [LWG2813]
Brian Bi. [std::function should not return dangling references](https://wg21.link/lwg2813). EWG. URL: <https://wg21.link/lwg2813>
- [N4268]
Richard Smith. [Allow constant evaluation for all non-type template arguments](https://wg21.link/n4268). 5 November 2014. URL: <https://wg21.link/n4268>
- [P0592R4]
Ville Voutilainen. [To boldly suggest an overall plan for C++23](https://wg21.link/p0592r4). 25 November 2019. URL: <https://wg21.link/p0592r4>
- [P0847r6]
Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. [Deducing this](https://wg21.link/p0847r6). 15 January 2021. URL: <https://wg21.link/p0847r6>
- [P0849r7]
Zhihao Yuan. [auto\(x\): decay-copy in the language](https://wg21.link/p0849r7). 15 February 2021. URL: <https://wg21.link/p0849r7>
- [P1000R4]
Herb Sutter. [C++ IS schedule](https://wg21.link/p1000r4). 14 February 2020. URL: <https://wg21.link/p1000r4>
- [P1018r9]
JF Bastien. [C++ Language Evolution status - pandemic edition - 2021/01–2021/03](https://wg21.link/p1018r9). 8 March 2021. URL: <https://wg21.link/p1018r9>
- [P1371R3]
Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. [Pattern Matching](https://wg21.link/p1371r3). 15 September 2020. URL: <https://wg21.link/p1371r3>

- [P1393R0]
D. S. Hollman, Chris Kohlhoff, Bryce Lebach, Jared Hoberock, Gordon Brown, Michał Dominiak. [A General Property Customization Mechanism](https://wg21.link/p1393r0). 13 January 2019. URL: <https://wg21.link/p1393r0>
- [P1401R3]
Andrzej Krzemiński. [Narrowing contextual conversions to bool](https://wg21.link/p1401r3). 27 May 2020. URL: <https://wg21.link/p1401r3>
- [P1467R4]
David Olsen, Michał Dominiak. [Extended floating-point types and standard names](https://wg21.link/p1467r4). 14 June 2020. URL: <https://wg21.link/p1467r4>
- [P1701R1]
Nathan Sidwell. [Inline Namespaces: Fragility Bites](https://wg21.link/p1701r1). 13 September 2020. URL: <https://wg21.link/p1701r1>
- [P1726R3]
Paul E. McKenney, Maged Michael, Jens Mauer, Peter Sewell, Martin Uecker, Hans Boehm, Hubert Tong, Niall Douglas, Will Deacon, Michael Wong, and David Goldblatt. [Pointer lifetime-end zap](https://wg21.link/p1726r3). 21 February 2020. URL: <https://wg21.link/p1726r3>
- [P1858R2]
Barry Revzin. [Generalized pack declaration and usage](https://wg21.link/p1858r2). 1 March 2020. URL: <https://wg21.link/p1858r2>
- [P1875R0]
Michael Spear, Hans Boehm, Victor Luchangco, Michael Scott, Michael Wong. [Transactional Memory Lite Support in C++](https://wg21.link/p1875r0). 7 October 2019. URL: <https://wg21.link/p1875r0>
- [P1938R1]
Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. [if consteval](https://wg21.link/p1938r1). 2 March 2020. URL: <https://wg21.link/p1938r1>
- [P1949R4]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. [C++ Identifier Syntax using Unicode Standard Annex 31](https://wg21.link/p1949r4). 5 June 2020. URL: <https://wg21.link/p1949r4>
- [P1949R6]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. [C++ Identifier Syntax using Unicode Standard Annex 31](https://wg21.link/p1949r6). 15 September 2020. URL: <https://wg21.link/p1949r6>
- [P1967R2]
JeanHeyd Meneide. [#embed - a simple, scannable preprocessor-based resource acquisition method](https://wg21.link/p1967r2). 2 March 2020. URL: <https://wg21.link/p1967r2>
- [P1988r1]
Steve Downey. [Allow Templates in Local Classes](https://wg21.link/p1988r1). 29 February 2020. URL: <https://wg21.link/p1988r1>
- [P1999R0]
Ville Voutilainen. [Process proposal: double-check evolutionary material via a Tentatively Ready status](https://wg21.link/p1999r0). 25 November 2019. URL: <https://wg21.link/p1999r0>
- [P2011R1]
Barry Revzin, Colby Pike. [A pipeline-rewrite operator](https://wg21.link/p2011r1). 16 April 2020. URL: <https://wg21.link/p2011r1>
- [P2013R2]
Ben Craig. [Freestanding Language: Optional ::operator new](https://wg21.link/p2013r2). 14 August 2020. URL: <https://wg21.link/p2013r2>
- [P2025R1]
Anton Zhilin. [Guaranteed copy elision for return variables](https://wg21.link/p2025r1). 15 June 2020. URL: <https://wg21.link/p2025r1>
- [P2044R0]
Robert Leahy. [Member Templates for Local Classes](https://wg21.link/p2044r0). 12 January 2020. URL: <https://wg21.link/p2044r0>
- [P2044r2]
Robert Leahy. [Member Templates for Local Classes](https://wg21.link/p2044r2). 14 April 2020. URL: <https://wg21.link/p2044r2>
- [P2139R1]
Alisdair Meredith. [Reviewing Deprecated Facilities of C++20 for C++23](https://wg21.link/p2139r1). 15 June 2020. URL: <https://wg21.link/p2139r1>
- [P2139R2]
Alisdair Meredith. [Reviewing Deprecated Facilities of C++20 for C++23](https://wg21.link/p2139r2). 15 July 2020. URL: <https://wg21.link/p2139r2>
- [P2145R1]
Bryce Adelstein Lebach, Titus Winters, Fabio Fracassi, Billy Baker, Nevin Liber, JF Bastien, David Stone, Botond Ballo, Tom Honermann. [Evolving C++ Remotely](https://wg21.link/p2145r1). 15 September 2020. URL: <https://wg21.link/p2145r1>

[P2156R0]

Erich Keane. [Allow Duplicate Attributes](https://wg21.link/p2156r0). 15 April 2020. URL: <https://wg21.link/p2156r0>

[P2173R0]

Daveed Vandevor, Inbal Levi, Ville Voutilainen. [Attributes on Lambda-Expressions](https://wg21.link/p2173r0). 15 May 2020. URL: <https://wg21.link/p2173r0>

[P2174R0]

Zhihao Yuan. [Compound Literals](https://wg21.link/p2174r0). 16 May 2020. URL: <https://wg21.link/p2174r0>

[P2186R0]

JF Bastien, Alisdair Meredith. [Removing Garbage Collection Support](https://wg21.link/p2186r0). 12 July 2020. URL: <https://wg21.link/p2186r0>

[P2195R0]

Bryce Adelstein LeLach. [Electronic Straw Polls](https://wg21.link/p2195r0). 15 September 2020. URL: <https://wg21.link/p2195r0>

[P2201R0]

Jens Maurer. [Mixed string literal concatenation](https://wg21.link/p2201r0). 14 July 2020. URL: <https://wg21.link/p2201r0>

[P2223R0]

Corentin Jabot. [Trimming whitespaces before line splicing](https://wg21.link/p2223r0). 14 September 2020. URL: <https://wg21.link/p2223r0>

[P2242r2]

Ville Voutilainen. [Non-literal variables \(and labels and gotos\) in constexpr functions](https://wg21.link/p2242r2). 15 March 2021. URL: <https://wg21.link/p2242r2>